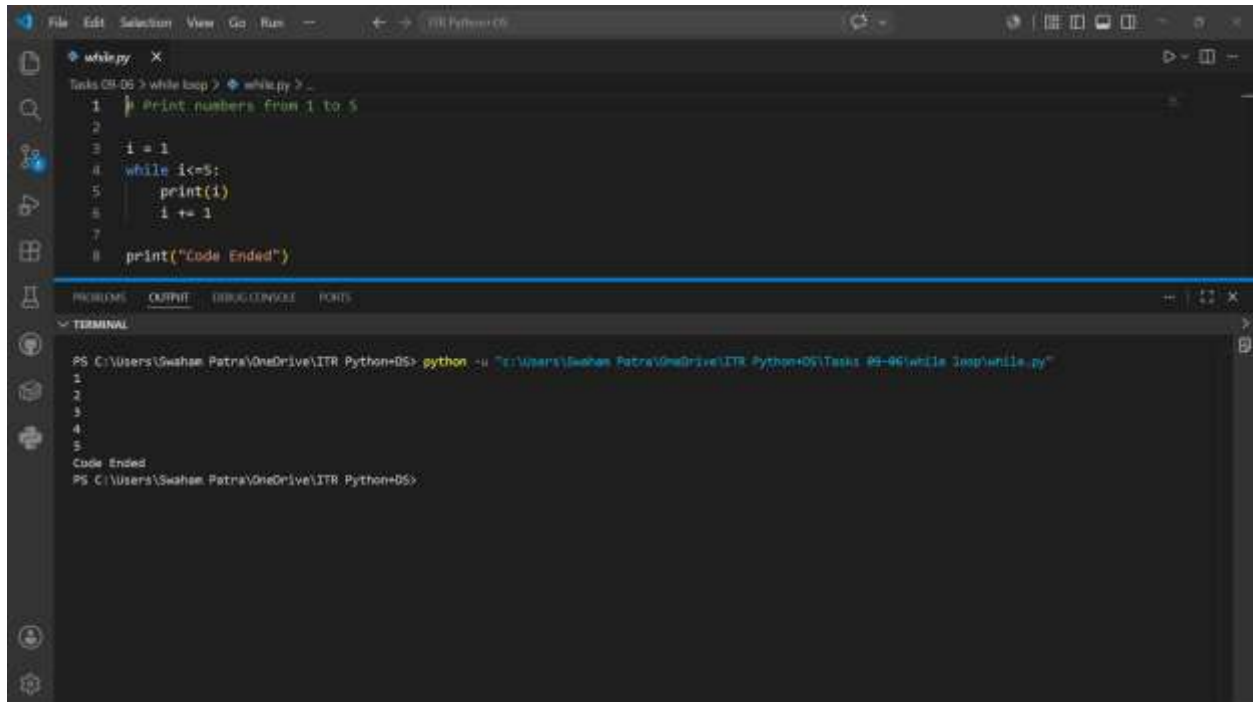


Swaham Pitambar Patra

✓ Basic While Loop



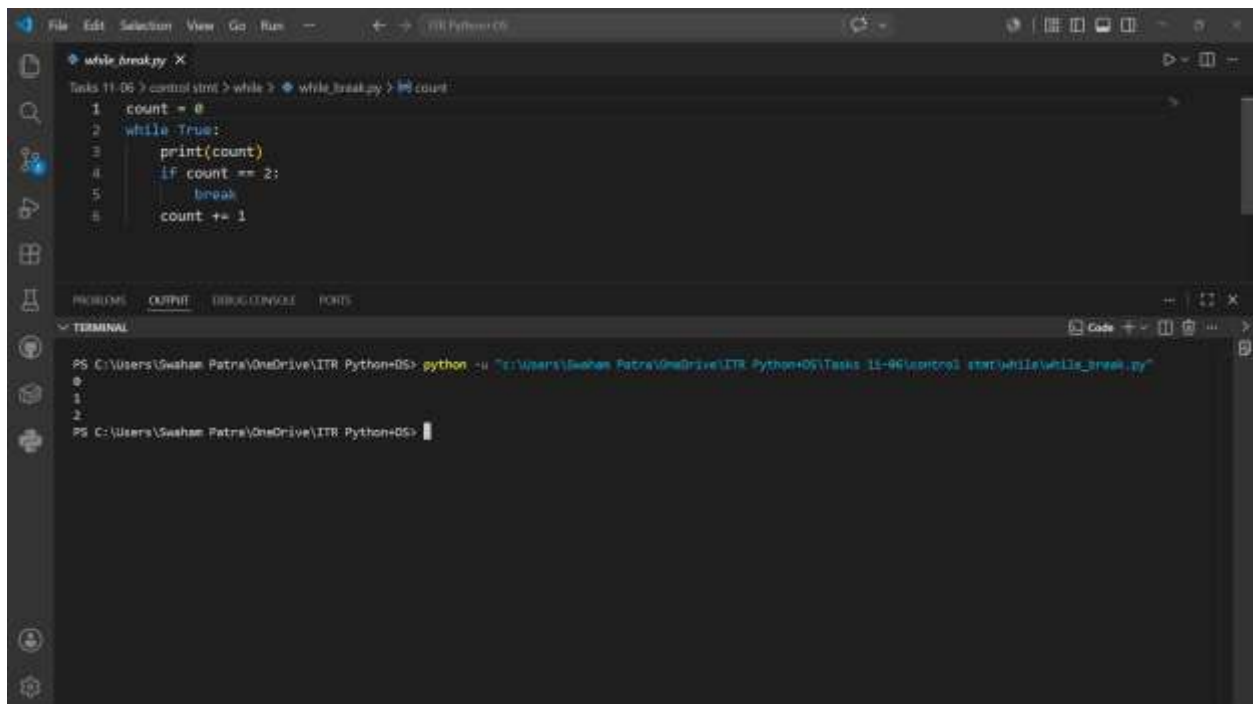
The screenshot shows a Visual Studio Code editor with a file named `while.py`. The code in the editor is as follows:

```
1 | Print numbers from 1 to 5
2 |
3 | i = 1
4 | while i<=5:
5 |     print(i)
6 |     i += 1
7 |
8 | print("Code Ended")
```

The terminal window at the bottom shows the command `python -u "c:\Users\Swaham Patra\OneDrive\ITR Python\DS\Tasks 09-06\while loop\while.py"` being executed. The output of the program is:

```
1
2
3
4
5
Code Ended
PS C:\Users\Swaham Patra\OneDrive\ITR Python\DS>
```

✓ While with Break



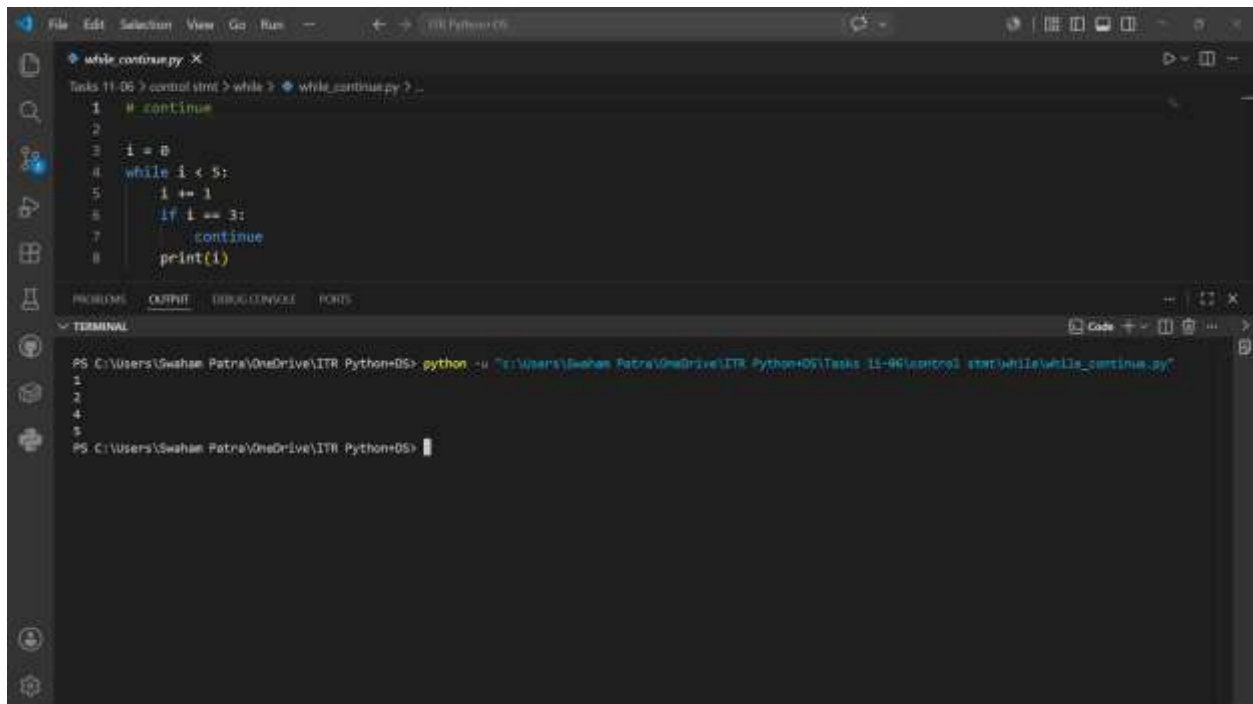
The screenshot shows a Visual Studio Code editor with a file named `while_break.py`. The code in the editor is as follows:

```
1 | count = 0
2 | while True:
3 |     print(count)
4 |     if count == 2:
5 |         break
6 |     count += 1
```

The terminal window at the bottom shows the command `python -u "c:\Users\Swaham Patra\OneDrive\ITR Python\DS\Tasks 11-06\control stmt\while\while_break.py"` being executed. The output of the program is:

```
0
1
2
PS C:\Users\Swaham Patra\OneDrive\ITR Python\DS>
```

✓ While with Continue



The screenshot shows a VS Code editor with a file named `while_continue.py`. The code is as follows:

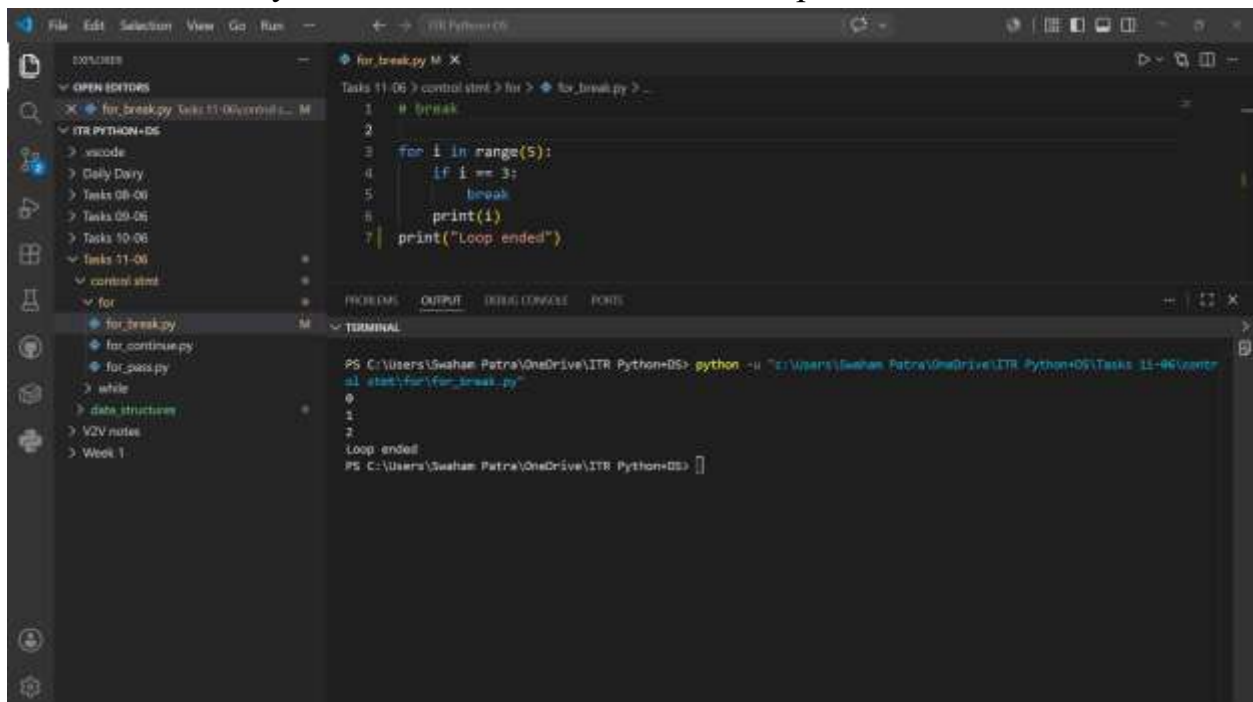
```
1 # continue
2
3 i = 0
4 while i < 5:
5     i += 1
6     if i == 3:
7         continue
8     print(i)
```

The terminal output shows the execution of the script, which prints the numbers 1, 2, 4, and 5, skipping the number 3.

```
PS C:\Users\Swaham Patra\OneDrive\ITR Python> python -u "c:\Users\Swaham Patra\OneDrive\ITR Python\06\Tasks 11-06\control stmt\while\while_continue.py"
1
2
4
5
PS C:\Users\Swaham Patra\OneDrive\ITR Python>
```

✓ Use of Break

- Used to break out of the loop after a particular condition is satisfied
And directly execute the next line after the loop



The screenshot shows a VS Code editor with a file named `for_break.py`. The code is as follows:

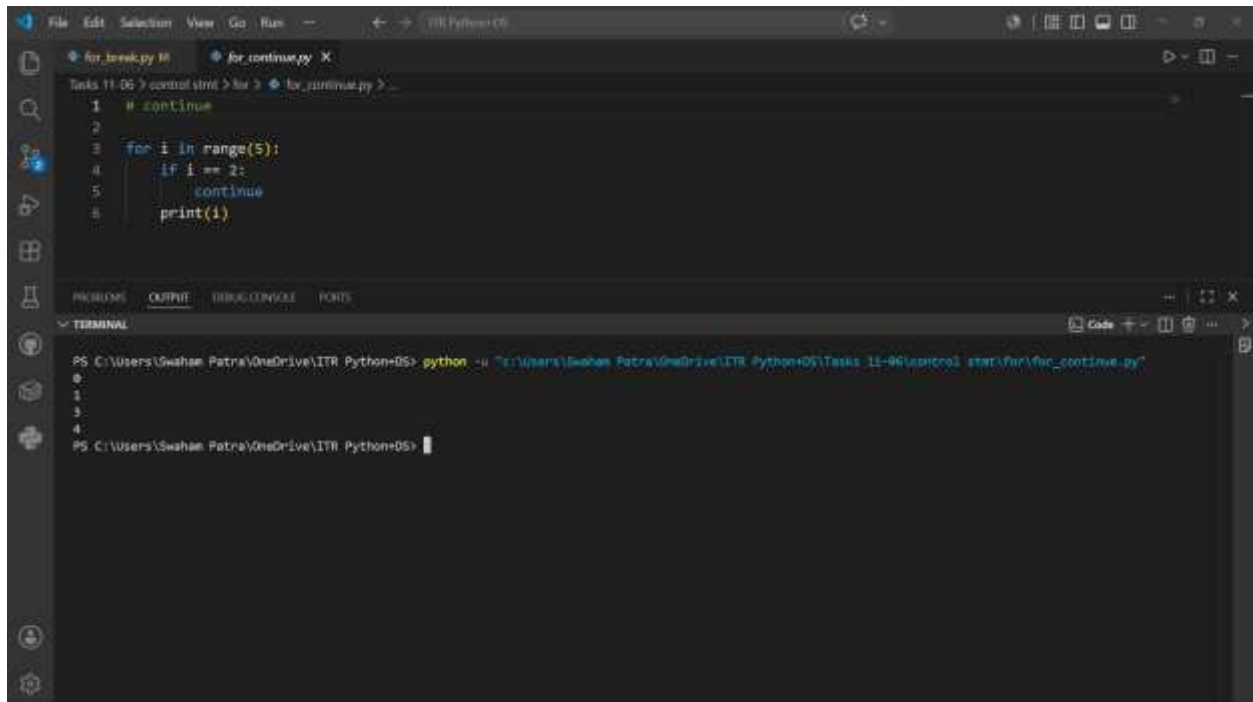
```
1 # break
2
3 for i in range(5):
4     if i == 3:
5         break
6     print(i)
7 print("Loop ended")
```

The terminal output shows the execution of the script, which prints the numbers 0, 1, and 2, followed by "Loop ended".

```
PS C:\Users\Swaham Patra\OneDrive\ITR Python> python -u "c:\Users\Swaham Patra\OneDrive\ITR Python\06\Tasks 11-06\control stmt\for\for_break.py"
0
1
2
Loop ended
PS C:\Users\Swaham Patra\OneDrive\ITR Python>
```

✓ Use of Continue

- Used to skip an iteration in the loop and the rest of the loop runs until the stopping point justified.



The screenshot shows a Visual Studio Code editor with a Python file named `for_continue.py`. The code is as follows:

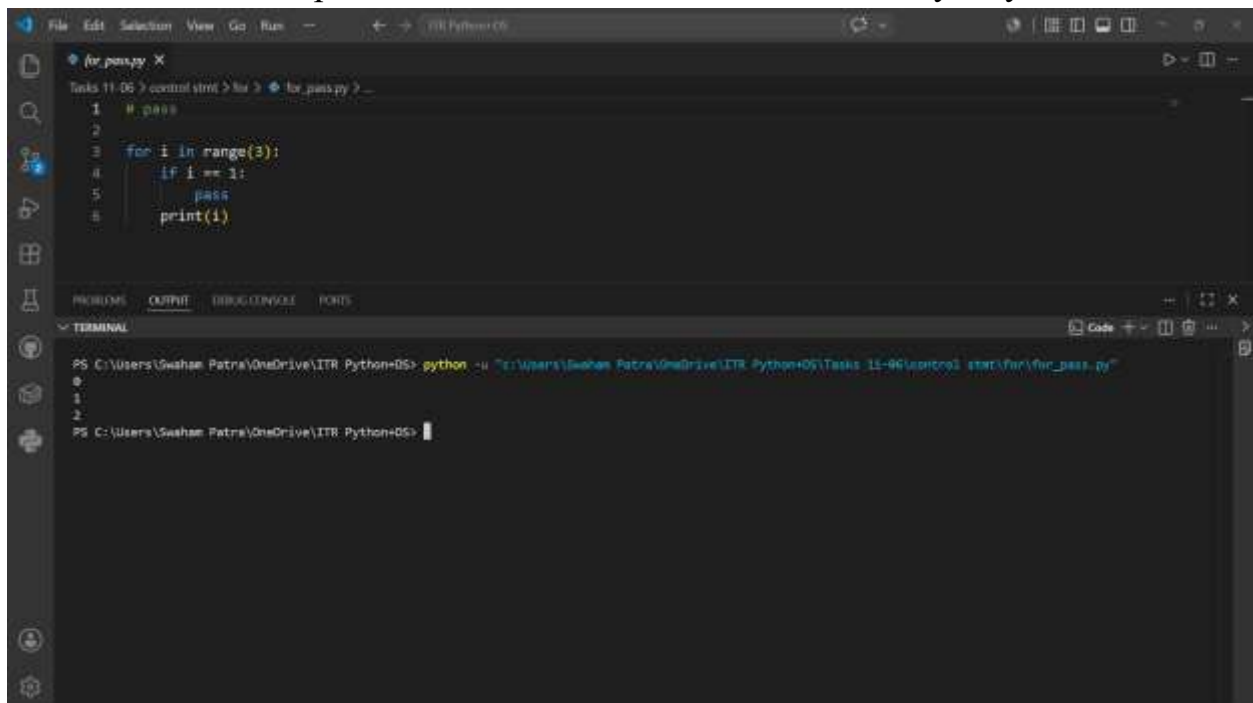
```
1 # continue
2
3 for i in range(5):
4     if i == 2:
5         continue
6     print(i)
```

The terminal window at the bottom shows the command `python -u "C:\Users\Swaham Patra\OneDrive\ITR Python\DS\Tasks 11-96\script2_start\for\for_continue.py"` being executed. The output of the script is:

```
0
1
3
4
```

✓ Use of Pass

- Just used as a placeholder. Doesn't affect the code in any way.



The screenshot shows a Visual Studio Code editor with a Python file named `for_pass.py`. The code is as follows:

```
1 # pass
2
3 for i in range(3):
4     if i == 1:
5         pass
6     print(i)
```

The terminal window at the bottom shows the command `python -u "C:\Users\Swaham Patra\OneDrive\ITR Python\DS\Tasks 11-96\script2_start\for\for_pass.py"` being executed. The output of the script is:

```
0
1
2
```

✓ Use of List

- Lists are used when there is a need to make a collection of heterogeneous data in a single variable and is to be mutable.

Eg. List1 = ["person1",.....]

✓ Use of Tuple

- Tuples are used when there is a need of heterogeneous data to be in a single variable and is immutable.

Eg. Tuple1 = {10, 20,.....}

✓ Use of Sets

- Sets are used when there is a requirement of only unique heterogeneous data.

Eg. Set1 = {122, 2, 234, 54,.....}

✓ use of Dictionary

- Dictionaries are used when data is required in pairs of key-value. When heterogeneous data must be in relation with each other.

Eg. Dict1 = {"key1":"value1", "key2":"value2",.....}

✓ Common method of all data structures

#List

##Code

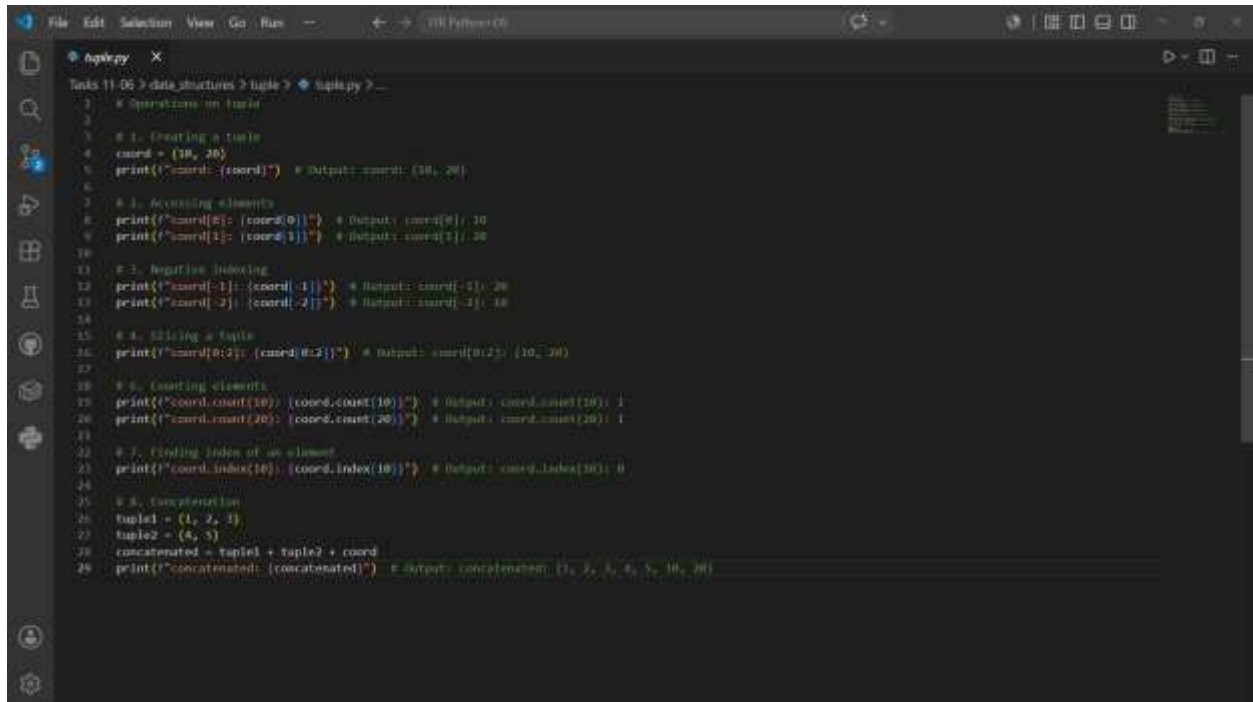
```
File Edit Selection View Go Run - Python 3.8
list.py
Tasks 11-06 > data_structures > list > list.py > ...
1 # Operations on lists
2
3 # 1. Creating a list
4 fruits = ['apple', 'banana', 'cherry']
5 print(f'fruits: {fruits}') # Output: fruits: ['apple', 'banana', 'cherry']
6
7 # 2. Accessing elements
8 print(f'fruits[0]: {fruits[0]}') # Output: fruits[0]: apple
9 print(f'fruits[1]: {fruits[1]}') # Output: fruits[1]: banana
10 print(f'fruits[2]: {fruits[2]}') # Output: fruits[2]: cherry
11
12 # 3. Slicing a list
13 print(f'fruits[0:2]: {fruits[0:2]}') # Output: fruits[0:2]: ['apple', 'banana']
14 print(f'fruits[1:3]: {fruits[1:3]}') # Output: fruits[1:3]: ['banana', 'cherry']
15
16 # 4. Negative indexing
17 print(f'fruits[-1]: {fruits[-1]}') # Output: fruits[-1]: cherry
18 print(f'fruits[-2]: {fruits[-2]}') # Output: fruits[-2]: banana
19 print(f'fruits[-3]: {fruits[-3]}') # Output: fruits[-3]: apple
20
21 # 5. Changing element
22 fruits[0] = 'orange'
23 print(f'fruits: {fruits}') # Output: fruits: ['orange', 'banana', 'cherry']
24
25 # 6. Adding elements with append()
26 fruits.append('orange')
27 print(f'fruits: {fruits}') # Output: fruits: ['orange', 'banana', 'cherry', 'orange']
28
29 # 7. Inserting elements
30 fruits.insert(1, 'kiwi')
31 print(f'fruits: {fruits}') # Output: fruits: ['orange', 'kiwi', 'banana', 'cherry', 'orange']
32
33 # 8. Removing elements
34 fruits.remove('banana')
35 print(f'fruits: {fruits}') # Output: fruits: ['orange', 'kiwi', 'cherry', 'orange']
```

##Output

```
File Edit Selection View Go Run - Python 3.8
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL
PS C:\Users\Saaham Patra\OneDrive\ITR Python\OS> python -u "C:\Users\Saaham Patra\OneDrive\ITR Python\OS\Tasks 11-06\data_structures\list\list.py"
fruits: ['apple', 'banana', 'cherry']
fruits[0]: apple
fruits[1]: banana
fruits[2]: cherry
fruits[0:2]: ['apple', 'banana']
fruits[1:3]: ['banana', 'cherry']
fruits[-1]: cherry
fruits[-2]: banana
fruits[-3]: apple
fruits: ['orange', 'banana', 'cherry']
fruits: ['orange', 'banana', 'cherry', 'orange']
fruits: ['orange', 'kiwi', 'banana', 'cherry', 'orange']
fruits: ['orange', 'kiwi', 'cherry', 'orange']
PS C:\Users\Saaham Patra\OneDrive\ITR Python\OS>
```

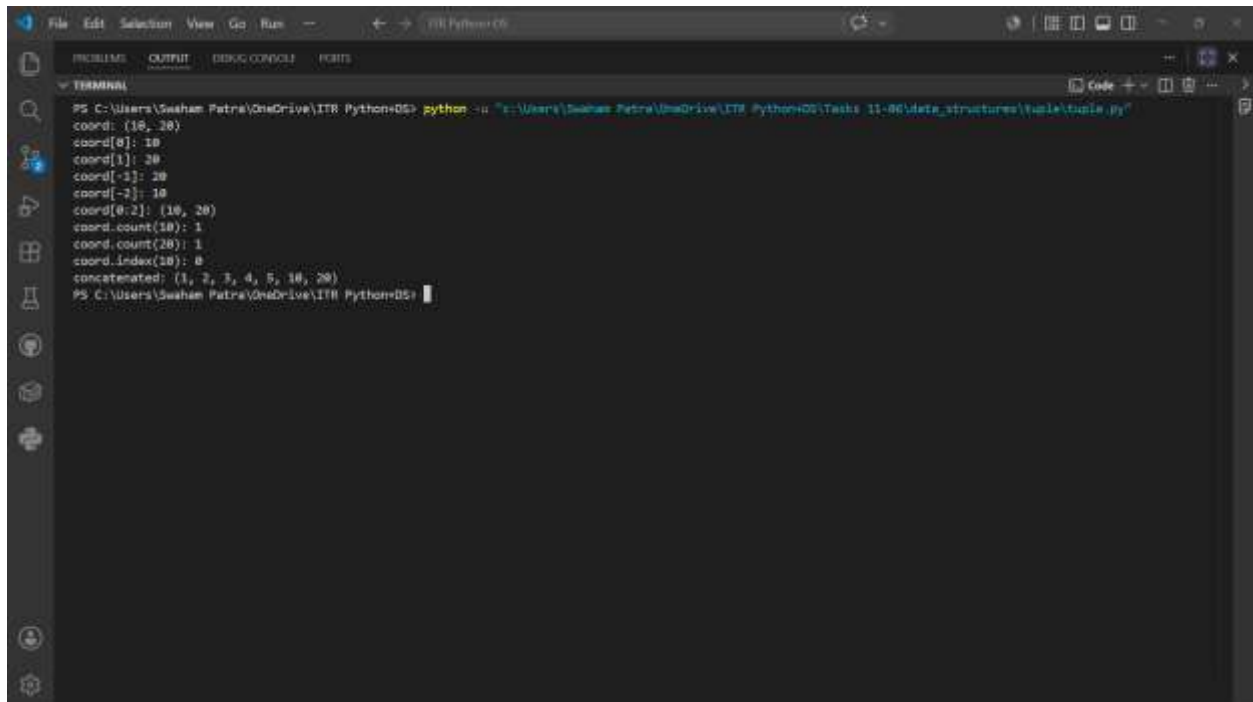
#Tuple

##Code



```
File Edit Selection View Go Run - Python+CS
tuple.py
Tasks 11-06 > data structures > tuple > tuple.py
1 # Operations on tuple
2
3 # 1. Creating a tuple
4 coord = (10, 20)
5 print(f"coord: {coord}") # Output: coord: (10, 20)
6
7 # 2. Accessing elements
8 print(f"coord[0]: {coord[0]}") # Output: coord[0]: 10
9 print(f"coord[1]: {coord[1]}") # Output: coord[1]: 20
10
11 # 3. Negative indexing
12 print(f"coord[-1]: {coord[-1]}") # Output: coord[-1]: 20
13 print(f"coord[-2]: {coord[-2]}") # Output: coord[-2]: 10
14
15 # 4. Slicing a tuple
16 print(f"coord[0:2]: {coord[0:2]}") # Output: coord[0:2]: (10, 20)
17
18 # 5. Counting elements
19 print(f"coord.count(10): {coord.count(10)}") # Output: coord.count(10): 1
20 print(f"coord.count(20): {coord.count(20)}") # Output: coord.count(20): 1
21
22 # 6. Finding index of an element
23 print(f"coord.index(10): {coord.index(10)}") # Output: coord.index(10): 0
24
25 # 7. Concatenation
26 tuple1 = (1, 2, 3)
27 tuple2 = (4, 5)
28 concatenated = tuple1 + tuple2 + coord
29 print(f"concatenated: {concatenated}") # Output: concatenated: (1, 2, 3, 4, 5, 10, 20)
```

##Output



```
File Edit Selection View Go Run - Python+CS
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL
PS C:\Users\Saaham Patra\OneDrive\ITR Python+CS> python -u "c:\Users\Saaham Patra\OneDrive\ITR Python+CS\Tasks 11-06\data_structures\tuple\tuple.py"
coord: (10, 20)
coord[0]: 10
coord[1]: 20
coord[-1]: 20
coord[-2]: 10
coord[0:2]: (10, 20)
coord.count(10): 1
coord.count(20): 1
coord.index(10): 0
concatenated: (1, 2, 3, 4, 5, 10, 20)
PS C:\Users\Saaham Patra\OneDrive\ITR Python+CS>
```

#Sets

##Code

```
File Edit Selection View Go Run - ITR Python+DS
set.py
Tasks 11-06 > data structures > sets > set.py > ...
1 # Operations on set
2
3 # 1. Creating a set
4 from typing import Union
5
6 nums = {1, 2, 3, 4}
7 print(f'nums: {nums}') # Output: nums: {1, 2, 3, 4}
8
9
10 # 2. Adding elements
11 nums.add(5)
12 print(f'nums after adding 5: {nums}') # Output: nums after adding 5: {1, 2, 3, 4, 5}
13
14 # 3. Removing elements
15 nums.remove(2)
16 print(f'nums after removing 2: {nums}') # Output: nums after removing 2: {1, 3, 4, 5}
17
18 # 4. Union
19 set1 = {1, 2, 3}
20 set2 = {3, 4, 5}
21 print(f'set1: {set1} set2: {set2} Union: {set1 | set2}') # Output: Union: {1, 2, 3, 4, 5}
22
23 # 5. Intersection
24 print(f'set1: {set1} set2: {set2} Intersection: {set1 & set2}') # Output: Intersection: {3}
25
26 # 6. Difference
27 print(f'set1: {set1} set2: {set2} Difference (set1 - set2): {set1 - set2}') # Output: Difference (set1 - set2): {1, 2}
28 print(f'set1: {set1} set2: {set2} Difference (set2 - set1): {set2 - set1}') # Output: Difference (set2 - set1): {4, 5}
29
30 # 7. Membership
31 num1 = {1, 2, 3, 4}
32 print(f'Membership: 2 in num1: {2 in num1}') # Output: Membership: 2 in num1: True
```

##Output

```
File Edit Selection View Go - ITR Python+DS
PROBLEMS OUTPUT DEBUG CONSOLE FORKS
TERMINAL
PS C:\Users\Swahan Patra\OneDrive\ITR Python+DS> python -u "c:\Users\Swahan Patra\OneDrive\ITR Python+DS\Tasks 11-06\data_structure\sets\set.py"
nums: {1, 2, 3, 4}
nums after adding 5: {1, 2, 3, 4, 5}
nums after removing 2: {1, 3, 4, 5}
set1:{1, 2, 3} set2:{3, 4, 5} Union: {1, 2, 3, 4, 5}
set1:{1, 2, 3} set2:{3, 4, 5} Intersection: {3}
set1:{1, 2, 3} set2:{3, 4, 5} Difference (set1 - set2): {1, 2}
set1:{1, 2, 3} set2:{3, 4, 5} Difference (set2 - set1): {4, 5}
Membership: 2 in num1: True
PS C:\Users\Swahan Patra\OneDrive\ITR Python+DS>
```

#Dictionaries

##Code

```
File Edit Selection View Go Run - Python 3.8
dictionary.py X
Tasks 11-06 > data structures > dictionary > dictionary.py > ...
1 # Operations on Dictionaries
2
3 # 1. Creating a Dictionary
4 person = {"name": "Swaham", "age": 17}
5 print(person)
6
7 # 2. Accessing values
8 print(person["name"]) # Output: Swaham
9 print(person["age"]) # Output: 17
10
11 # 3. Adding a new key-value pair
12 person["city"] = "Dombivli"
13 print(person)
14
15 # 4. Changing values
16 person["age"] = 18
17 print(person)
18
19 # 5. Removing a key-value pair
20 person.pop("city")
21 print(person)
22
23 # 6. Keys and Values
24 print(person.keys()) # Output: dict_keys(['name', 'age'])
25 print(person.values()) # Output: dict_values(['Swaham', 18])
26
27 # 7. Checking key existence
28 print("Checking if 'name' exists in person? {'name' in person}") # Output: True
```

##Output

```
File Edit Selection View Go Run - Python 3.8
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL
PS C:\Users\Swaham Patra\OneDrive\ITR Python> python -u "c:\Users\Swaham Patra\OneDrive\ITR Python\Tasks 11-06\data_structures\Dictionary\dictionary.py"
{'name': 'Swaham', 'age': 17}
Swaham
17
{'name': 'Swaham', 'age': 17, 'city': 'Dombivli'}
{'name': 'Swaham', 'age': 18, 'city': 'Dombivli'}
{'name': 'Swaham', 'age': 18}
dict_keys(['name', 'age'])
dict_values(['Swaham', 18])
Checking if 'name' exists in person? True
PS C:\Users\Swaham Patra\OneDrive\ITR Python>
```